ANHANGUERA

COMPUTAÇÃO EM NUVEM

Cadastro Acadêmico Cloud

Banco de dados em nuvem com controle de acesso, ingestão e recuperação de dados

Instituição: Anhanguera

Disciplina: Computação em Nuvem

Integrante informado: Marcus

Demais integrantes: _____

Professor(a): _____

Turma/unidade: _____

Data: 11 de novembro de 2026

1. Objetivo e atendimento à atividade

O objetivo foi implementar um banco de dados remoto para cadastro de alunos, com controle de acesso e cópia de segurança. Como ampliação, foi desenvolvida uma pipeline de dados em Python, orquestrada pelo Apache Airflow.

O banco PostgreSQL está hospedado no Supabase. A orquestração funciona localmente em containers Linux no Docker Desktop com WSL2. Portanto, os dados acadêmicos estão na nuvem, enquanto o painel e o executor do Airflow estão no computador de desenvolvimento.

Requisito	Implementação	Evidência
Banco remoto	Supabase/PostgreSQL	Tabelas consultadas pela conexão remota
Cadastro	Alunos, cursos e matrículas	50 alunos e 50 vínculos persistidos
Controle de acesso	Roles PostgreSQL e políticas RLS	Logins reais com operações permitidas e negadas
Cópia de segurança	pg_dump em formato custom	Arquivo e manifesto SHA-256 gerados
Recuperação	pg_restore em banco local separado	Contagens e hashes idênticos nas quatro tabelas

2. Arquitetura e implementação

Fluxo: Airflow -> API Random User -> validação -> DataFrames pandas -> transação PostgreSQL -> histórico da execução.

A API Random User fornece perfis fictícios brasileiros. A versão 1.4, a seed, a página e a quantidade de 50 perfis são fixas. O UUID do perfil identifica o aluno; curso e matrícula são simulados de forma determinística. Esses dados não representam alunos reais da instituição.

Componente	Responsabilidade
fonte.py / transformacao.py	Extrair os perfis e validar os dados
quadros.py	Criar DataFrames e validar referências e duplicações
gravacao.py / pipeline.py	Resolver IDs, gravar o lote e registrar a execução
banco.py / acesso.py	Separar conexões e comprovar os perfis de acesso
backup.py	Exportar, restaurar e comparar os dados
dags/	Orquestrar ingestão e backup com novas tentativas

Tabela	Chave	Finalidade
alunos	id_aluno	Código de origem, nome, e-mail e datas
cursos	id_curso	Nome do curso e datas
matriculas	id_matricula	Vínculo aluno/curso, número, status e datas

Tabela	Chave	Finalidade
ingestoes	id_ingestao	Parâmetros, status, contagens, horários e erro

A carga usa UPSERT e preserva IDs. A repetição não duplica alunos ou vínculos, nem reativa matrículas canceladas. Alunos e cursos são gravados antes da resolução das chaves das matrículas. Os dados e a conclusão do histórico são confirmados na mesma transação; uma falha desfaz o lote.

3. Controle de acesso e comprovação

Foram criados os grupos academico_leitor, academico_operador e academico_pipeline, vinculados a três contas com LOGIN. As senhas foram geradas aleatoriamente e ficam somente no .env local. A ingestão passou a utilizar sua própria conta restrita, em vez da conta administrativa.

O leitor consulta as quatro tabelas; o operador pode cadastrar e alterar dados acadêmicos e consultar o histórico; a pipeline recebe as permissões necessárias à carga e ao histórico. Nenhum desses perfis pode excluir dados ou criar tabelas. As políticas permitem a leitura dos dados da aplicação aos perfis autorizados; não há segregação individual por aluno nesta versão.

Perfil	Alunos visíveis	Alterar aluno	Excluir / criar tabela
leitor	50	Bloqueado	Bloqueado / bloqueado
operador	50	Permitido	Bloqueado / bloqueado
pipeline	50	Permitido	Bloqueado / bloqueado

As verificações usaram conexões reais com as contas academico_consulta, academico_cadastro e academico_ingestor. A alteração permitida foi executada em transação desfeita ao final. As tentativas de exclusão e DDL retornaram falta de privilégio. As contas não são superusuárias, não criam roles ou bancos e não ignoram o RLS.

Foram confirmadas 12 políticas e RLS ativo nas quatro tabelas. Os papéis anon e authenticated permanecem sem acesso. O controle apresentado é de usuários do PostgreSQL; não foi implementada uma aplicação de cadastro com Supabase Auth. O login admin do Airflow protege apenas o painel de orquestração.

3.1 Logs e rastreabilidade

Os logs INFO registram etapas e contagens. Os logs ERROR identificam a etapa, a ingestão, a classe do erro, o SQLSTATE quando disponível e a pilha de execução. A mensagem bruta da exceção e os conteúdos dos perfis não são registrados. O histórico de falhas também fica em ingestoes.mensagem_erro quando o banco está acessível.

4. Backup e teste de restauração

A rotina gera um backup lógico do schema public da aplicação, incluindo as quatro tabelas, funções, restrições e políticas. Um snapshot exportado em transação REPEATABLE READ mantém o pg_dump e a comparação de origem no mesmo estado dos dados.

O arquivo custom é gravado no computador em backups/, fora do Git. Um manifesto registra a data, o hash SHA-256 do arquivo e a quantidade e hash dos registros de cada tabela. As credenciais e roles globais com senhas não fazem parte do dump; os privilégios são reaplicados pela migração versionada.

A recuperação foi feita com pg_restore em um banco novo no serviço Docker postgres-restauracao, separado do Supabase e sem porta publicada no host. Os perfis de plataforma necessários foram criados localmente sem LOGIN, e a migração de privilégios foi reaplicada. A origem não foi usada como destino de restauração.

Tabela	No backup	Restaurado	Comparação SHA-256
alunos	50	50	Idêntico
cursos	3	3	Idêntico
matriculas	50	50	Idêntico
ingestoes	6	6	Idêntico

Resultado final: APROVADO. RLS confirmado em 4 tabelas restauradas. A comparação considerou todas as colunas e linhas, ordenadas pela chave primária e serializadas de forma canônica em UTC.

Arquivo: academico_20261111_101228_be396d08.dump

Banco local recuperado: recuperacao_8438f9988e494909

```
SHA-256 do arquivo: 5cb57a2545f78c0f78a5b894e6a9640901cee388e91ab0301d2a465ab4b18436
```

Snapshot do backup: 2026-11-11T10:12:28.115415-03:00. Validação: 2026-11-11T10:17:09.895616-03:00.

O escopo é o banco da aplicação no schema public. Não se trata de uma cópia integral de todos os serviços Supabase, como Auth, Storage e configurações da plataforma. Os arquivos de backup ficam localmente e devem ser preservados em local seguro.

5. Evidências das execuções

DAG	Execução de validação	Estado final
cadastro_academico_supabase	validacao_acesso_001	success
backup_academico_supabase	validacao_backup_001	success

A execução validacao_acesso_001 comprova a ingestão com a conta restrita. A execução validacao_backup_001 comprova as tarefas de exportação e restauração. As DAGs usam acionamento manual; o agendamento periódico pode ser definido conforme a necessidade.

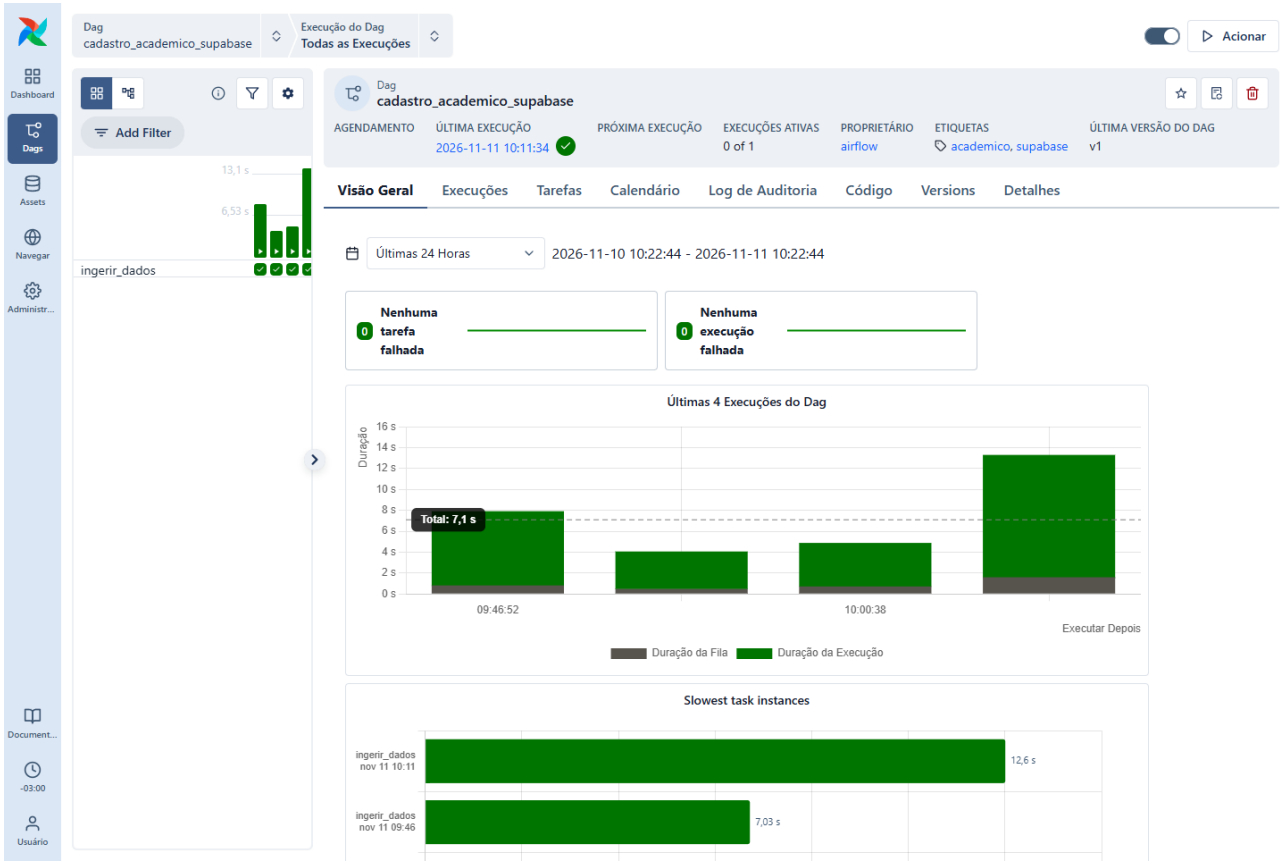


Figura 1. DAG de ingestão no Airflow local.

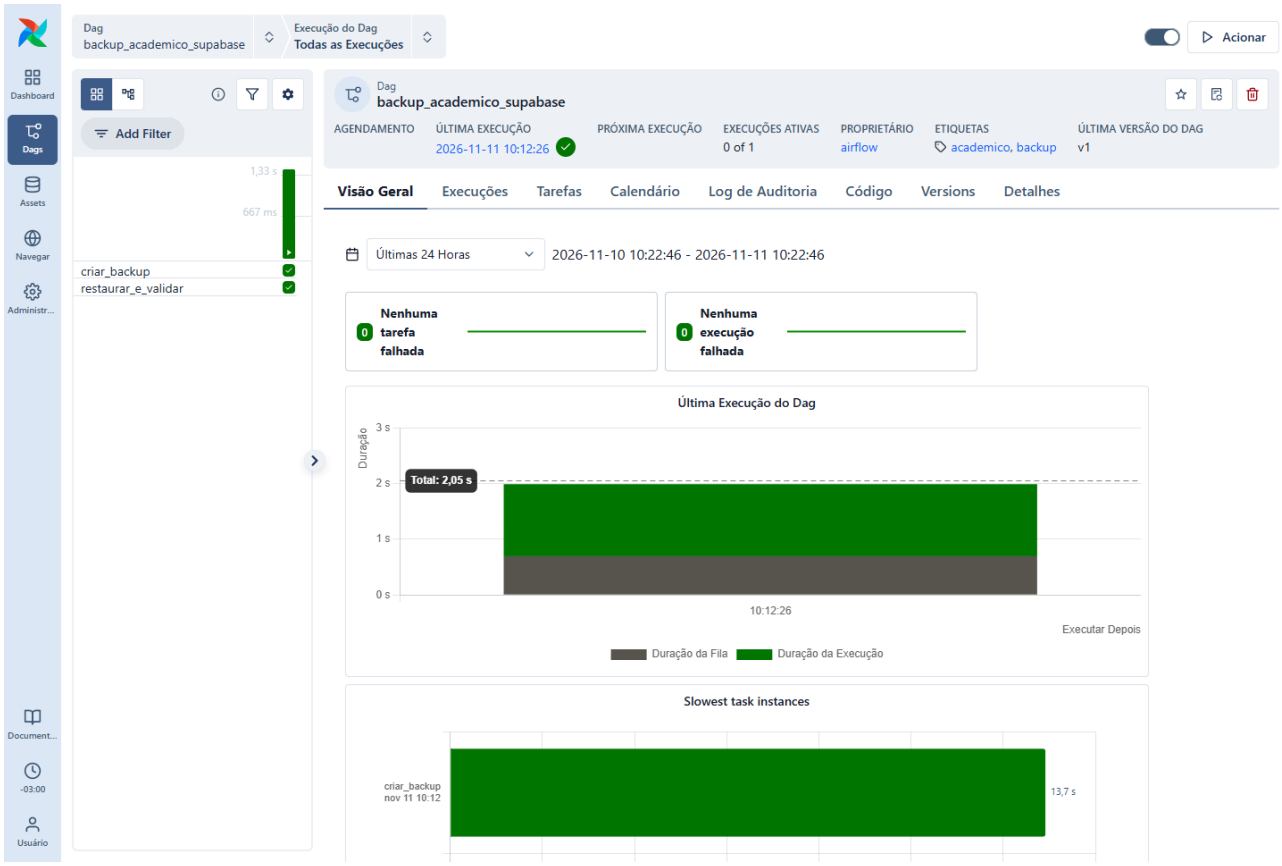


Figura 2. DAG de backup e restauração no Airflow local.

As evidências estruturadas estão em entrega/evidencias.json. O manifesto do backup contém os hashes completos de cada tabela para repetição da comparação. As capturas não exibem senhas nem tokens.

6. Conclusão

A implementação atende ao cadastro remoto de dados fictícios, ao controle de acesso por perfis e à cópia de segurança com recuperação comprovada. A ingestão e a validação do backup são reproduzíveis pelo Airflow. A restauração foi verificada pela igualdade das contagens e dos hashes das quatro tabelas.

Antes da entrega, preencher professor, turma/unidade e os demais integrantes na capa. Como evoluções, podem ser adotados agendamento periódico, armazenamento adicional dos backups e uma interface de cadastro para usuários finais.

7. Referências

SUPABASE. PostgreSQL Roles. https://supabase.com/docs/guides/database/postgres/roles. Acesso em: 11 nov. 2026.

SUPABASE. Database Backups. https://supabase.com/docs/guides/platform/backups. Acesso em: 11 nov. 2026.

POSTGRESQL. pg_dump. https://www.postgresql.org/docs/current/app-pgdump.html. Acesso em: 11 nov. 2026.

APACHE AIRFLOW. Quick Start. https://airflow.apache.org/docs/apache-airflow/stable/start.html. Acesso em: 11 nov. 2026.

RANDOM USER. Documentation. https://randomuser.me/documentation. Acesso em: 11 nov. 2026.